

데이터 명세 · 분석 · 전처리

- 프로젝트: SKN31-2nd-4Team — KKBox 이탈 예측 (Churn Prediction)
- 데이터 출처: [Kaggle KKBox Churn Prediction Challenge](#)
- 참고 논문: *KKBox Churn Rate Prediction: A Special Focus on Imbalanced Dataset* (Xu et al., 2017)

1. 데이터 설명

1-1. 원본 데이터

KKBox 챌린지는 동일 형식의 데이터를 시점별로 v1 / v2 / v3로 나누어 제공한다. 본 프로젝트는 2017년 3월 이탈을 예측 대상으로 하므로, 시점 적합성에 맞춰 4개 파일만 사용한다.

#	원본 파일	설명	시점	사용 여부
1	train	사용자 ID + 이탈 여부	2017년 2월 이탈	✗
2	train_v2	동일 형식	2017년 3월 이탈	✓
3	transactions	거래 내역	~2017.02.28	✓ (결합)
4	transactions_v2	거래 내역	2017.02.28~03.31	✓ (결합)
5	user_logs	일별 청취 로그	~2017.02.28	✗
6	user_logs_v2	일별 청취 로그	2017.02.28~03.31	✓
7	members	회원 정보	—	✗
8	members_v3	회원 정보 (만료일 컬럼 제거 정제판)	—	✓

원본 테이블 명세

train_data — 정답 레이블 (PK: msno)

컬럼명	타입	설명	비고
msno	String	회원 암호화 고유 ID	조인 키
is_churn	Int (Binary)	이탈 여부	1 =이탈, 0 =잔존 (Target)

members — 회원 인구통계 정보 (PK: msno)

컬럼명	타입	설명	비고
msno	String	회원 고유 ID	
city	Int (범주형)	거주 도시 코드	
bd	Int	나이	0 :범위 밖 값은 이상치
gender	String (범주형)	성별	male / female , 결측 존재
registered_via	Int (범주형)	가입 경로 코드	
registration_init_time	Int (YYYYMMDD)	최초 가입일	

transactions — 결제·구독 이력 (PK: msno , 1:N)

컬럼명	타입	설명	비고
msno	String	회원 고유 ID	
payment_method_id	Int (범주형)	결제 수단 코드	
payment_plan_days	Int	플랜 이용 기간(일)	예: 30, 120
plan_list_price	Int	정가 (NTD)	
actual_amount_paid	Int	실 결제액 (NTD)	
is_auto_renew	Int (Binary)	자동 갱신 여부	1 =자동, 0 =수동
transaction_date	Int (YYYYMMDD)	결제일	
membership_expire_date	Int (YYYYMMDD)	멤버십 만료 예정일	
is_cancel	Int (Binary)	구독 취소 여부	1 =취소, 0 =유지

user_logs — 일별 청취 행태 (PK: msno + date)

컬럼명	타입	설명	비고
msno	String	회원 고유 ID	
date	Int (YYYYMMDD)	로그 일자	
num_25	Int	25% 미만 청취 곡 수	
num_50	Int	25~50% 청취 곡 수	
num_75	Int	50~75% 청취 곡 수	
num_985	Int	75~98.5% 청취 곡 수	
num_100	Int	98.5~100% 청취 곡 수	완청
num_unq	Int	고유 곡 수	중복 제거된 곡수
total_secs	Float	당일 총 청취 시간(초)	

원본 데이터 규모 (사용 파일 기준)

테이블	사용 파일	행 수	컬럼 수
레이블	train_v2	970,960	2
회원	members_v3	6,769,473	6
거래	transactions + transactions_v2 (결합)	22,978,755	9
로그	user_logs_v2	18,396,362	9

1-2. 학습시킬 데이터셋 (최종 병합본)

msno 기준으로 4개 테이블을 병합한 결과로, 회원 1명당 1행이 되도록 거래·로그는 집계/최신 추출 후 결합한다.

- 행: train_v2 회원을 기준으로 inner join 하며 축소
- 컬럼: 42개 — msno (ID, 학습 제외) + is_churn (Target) + 피처 후보 40개
- ID·datetime·중복 컬럼은 모델링 단계에서 drop

#	컬럼명	출처	타입	설명
1	msno	train	str	회원 고유 ID (조인 키, 학습 제외)
2	is_churn	train	int	이탈 여부 — Target
3	city	members	int	거주 도시 코드
4	registered_via	members	int	가입 경로 (3,4,7,9 외 → 0)
5	registration_init_time	members	str	가입일 (학습제외 = 파생변수 이용)
6	reg_year	members	int	가입 연도 (파생)
7	reg_month	members	int	가입 월 (파생)
8	reg_day	members	int	가입 일 (파생)
9	reg_weekday	members	int	가입 요일 (파생)
10	reg_time_num	members	float	가입 시점 연속형 (파생)
11	gender_female	members	bool-> int	성별 더미 (여성)
12	gender_male	members	bool-> int	성별 더미 (남성)
13	gender_unknown	members	bool-> int	성별 더미 (결측→unknown)
14	payment_method_id	transactions	int	최근 거래 결제 수단
15	payment_plan_days	transactions	int	최근 거래 플랜 기간(일)
16	plan_list_price	transactions	int	최근 거래 정가 (NTD)
17	actual_amount_paid	transactions	int	최근 거래 실 결제액 (NTD)
18	is_auto_renew	transactions	int	자동 갱신 여부
19	transaction_date	transactions	str	최근 거래일 (학습제외 = 파생변수 이용)
20	membership_expire_date	transactions	str	멤버십 만료일 (학습제외 = 파생변수 이용)
21	is_cancel	transactions	int	취소 여부
22	trans_year	transactions	int	거래 연도 (파생)(학습제외 = 2017년도 데이터 밀집)
23	trans_month	transactions	int	거래 월 (파생)(학습제외 = 3월 데이터 밀집)
24	trans_day	transactions	int	거래 일 (파생)
25	trans_weekday	transactions	int	거래 요일 (파생)
26	expire_year	transactions	int	만료 연도 (파생)(학습제외 = 2017년도 데이터 밀집)
27	expire_month	transactions	int	만료 월 (파생)(학습제외 = 3월 데이터 밀집)
28	expire_day	transactions	int	만료 일 (파생)
29	expire_weekday	transactions	int	만료 요일 (파생)
30	pm_id_41	transactions	int	결제수단 41 여부 (최다, 파생)
31	pm_id_38	transactions	int	결제수단 38 여부 (2위, 파생)
32	pp_days_30	transactions	int	플랜 30일 여부 (최다, 파생)

#	컬럼명	출처	타입	설명
33	trans_count	transactions	int	유저별 총 거래 횟수 (파생)
34	total_secs	user_logs	float	총 청취 시간(초) 합 (집계)
35	num_25	user_logs	int	25% 청취 곡 수 합 (집계)
36	num_50	user_logs	int	50% 청취 곡 수 합 (집계)
37	num_75	user_logs	int	75% 청취 곡 수 합 (집계)
38	num_985	user_logs	int	98.5% 청취 곡 수 합 (집계)
39	num_100	user_logs	int	완청 곡 수 합 (집계)
40	num_unq	user_logs	int	고유 곡 수 합 (집계)
41	log_count	user_logs	int	유저별 총 로그 수 (파생)
42	last_listen_date	user_logs	int	마지막 청취일 (파생)(학습제외 = 파생변수 이용)

2. 데이터 분석 (EDA)

2-1. train_v2 (Target)

- 970,960행 / 2컬럼, 결측 없음
- 클래스 불균형: `is_churn = 0` → 883,630 (91.0%), `1` → 87,330 (9.0%) → 약 10:1
- 논문과 동일하게 소수 클래스(이탈) 비중이 작아, cost-sensitive 학습(`class_weight`)이 필요

2-2. members_v3

- 6,769,473행 / 6컬럼
- `gender` : 절반 이상 결측 → 삭제 대신 `unknown` 처리
- `bd` (나이): 이상치 67.2% (4,545,866건), 범위 `-7,168 ~ 2,016` (논문 4.1과 동일한 비정상 분포) → 컬럼 드랍
- `registered_via` : 3,4,7,9 에 집중, 그 외 코드는 소수 → 0 으로 통합

2-3. transactions (v1 + v2)

- 22,978,755행 / 9컬럼, 결측 0
- 고유 유저 2,426,143명, 유저당 거래 1~244회 (다수 유저는 소수 거래)
- 할인 발생: 867,020건 (3.8%) — `plan_list_price > actual_amount_paid`
- 이상치: `plan_list_price == 0` 1,516,957건 / `payment_plan_days == 0` 872,342건
- `membership_expire_date` : 9999년 등 이상치 6건 → 제거 (범위 `1970-01-01 ~ 2036-10-15`)
- `transaction_date` 범위: `2015-01-01 ~ 2017-03-31`
- 결제수단은 41 (1위)· 38 (2위), 플랜은 30일 이 최다 → 파생변수화

2-4. user_logs_v2

- 18,396,362행 / 9컬럼, 결측 없음
 - msno 기준 집계 후 유저 1,103,894명
 - 유저당 로그 수: 평균 16.7, 중앙값 18, 범위 1~31 (3월 한 달 = 최대 31일)
-

3. 데이터 전처리 (논문 기반)

3-1. 파일 선택 근거

- **train_v2** 만 사용 — 레이블이 2017년 3월 31일 만료 후 4월 갱신 여부를 기준으로 한 이탈이므로, 거래/로그의 v2 시점(2.28~3.31)의 가장 최근 결제값과 정합
- **transactions** + **transactions_v2** 결합 — 전체 거래 이력을 합쳐 최근 결제 1건을 단독 피처로 쓰고 나머지는 히수로 집계. 가장 많은 파생변수를 만드는 핵심 테이블
- **user_logs_v2** 만 사용 — 2.28~3.31 최근 한 달 청취 기록만 집계. 로그 없는 유저의 결측은 0으로 대체(= 최근 미청취 신호를 보존하기 위함)
- **members_v3** 사용 — 만료일 컬럼이 제거된 정제판

3-2. 테이블별 전처리

members_v3 (members_전처리.py)

1. gender 결측 → "unknown"
2. msno 중복 제거 (keep="last")
3. bd 드랍 (이상치 과다)
4. registration_init_time → datetime
5. 날짜 파생: reg_year, reg_month, reg_day, reg_weekday
6. 연속형 시점 파생: reg_time_num = reg_year + (reg_month-1)/12 + reg_day/365
7. registered_via: [3,4,7,9] 외 값 → 0
8. gender 원-핫 인코딩 → gender_female / gender_male / gender_unknown
9. 결과: 6 → 12컬럼

transactions (transaction_전처리.py)

1. 필요 9컬럼만 로드 + 메모리 최적화(dtype 다운캐스트)
2. transaction_date / membership_expire_date → datetime, 만료 연도 > 2030 행 제거
3. 날짜 파생: trans_* / expire_* (각 year/month/day/weekday)
4. 결제 파생: pm_id_41, pm_id_38, pp_days_30
5. 유저별 총 거래 히수 trans_count 계산
6. transaction_date 정렬 후 유저별 최신 거래 1건만 유지(drop_duplicates)
7. trans_count 병합 → 수치형 결측은 평균 대체
8. 결과: 9 → 21컬럼

user_logs_v2 (userlog_전처리.py)

1. 청크 단위 로드(대용량 대응)
2. date 정수화
3. msno 기준 groupby 집계:
4. 집계: total_secs , num_25 ~ num_100 , num_unq
5. 카운트: log_count (=로그 일수)
6. 최댓값: last_listen_date
7. 결과: 10컬럼 (유저 1행)

3-3. 병합 과정

msno 를 키로 순차 병합한다. 핵심 테이블은 inner join(결측 회원 제거), 로그는 left join 후 fillna(0) .

```
train_v2 (msno, is_churn)          ← 라벨 베이스
|   INNER JOIN (msno)              ← 양쪽 모두 존재하는 회원만 (결측 제거)
▼
+ members_v3 (전처리)
|   INNER JOIN (msno)              ← 거래 없는 회원 제거 (결측 제거)
▼
+ transactions (v1+v2 전처리, 최신 1건 + trans_count)
|   LEFT JOIN (msno) → fillna(0)   ← 로그 없는 유저 = 최근 미청취, 0으로 채워 정보 보존
▼
+ user_logs_v2 (집계)
▼
= 최종 데이터셋 (42컬럼: msno + is_churn + 피쳐 40)
```

3-4. 파생변수 요약

원본 대비 새로 생성한 변수는 약 22개다.

출처	파생변수
members	reg_year , reg_month , reg_day , reg_weekday , reg_time_num , gender_* (3)
transactions	trans_year/month/day/weekday , expire_year/month/day/weekday , pm_id_41 , pm_id_38 , pp_days_30 , trans_count
user_logs	log_count , last_listen_date 나머지 컬럼은 일별→유저별 집계 집계)